

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING



PROJECT REPORT
LSI Logic Design

A SIMPLE RISC CPU DESIGN

Advisor(s): Nguyen Thanh Loc

Student(s): Huynh Chanh Minh ID 2352736
Dang Thai Khang ID 2352464
Tran Nguyen Nam Hai ID 2252190
Dang Duy Nguyen ID 2352821

Ho Chi Minh City, May 7, 2026

Contents

1	Introduction	3
1.1	Project overview	3
1.2	Tools and equipment	3
1.2.1	Software	3
1.2.2	Hardware	4
1.3	Product features	4
2	Background	5
2.1	The RISC philosophy	5
2.2	Instruction execution model	5
2.3	Real-world relevance	6
3	System Design	7
3.1	Top-level block diagram	7
3.2	Functional blocks	7
3.2.1	Program Counter (PC)	7
3.2.2	Address Mux	8
3.2.3	Memory	8
3.2.4	Instruction Register (IR)	8
3.2.5	Accumulator Register (ACC)	8
3.2.6	ALU	8
3.2.7	Controller	9
4	Implementation	11
4.1	Project layout	11
4.1.1	Source code repository	11
4.2	Verilog modules	11
4.2.1	Top-level wrapper	11
4.2.2	Program Counter	12
4.2.3	Address Mux	13
4.2.4	Memory	13
4.2.5	Instruction Register and Accumulator	13
4.2.6	ALU	14
4.2.7	Controller	14
4.3	State-transition diagram	16
4.3.1	Instruction timing characteristics	16
4.3.2	Cycles per instruction (CPI)	17
4.4	Per-instruction execution flow	18
4.5	Assembler helper	18

5	Verification and Evaluation	20
5.1	Verification methodology	20
5.2	Test program 1 — JMP / LDA / SKZ / STO / XOR	20
5.3	Test program 2 — AND / ADD	21
5.4	Test program 3 — Fibonacci	22
5.5	Performance evaluation	23
5.5.1	Synthesis setup	23
5.5.2	Area result	24
5.5.3	Timing result	25
5.5.4	Power result	25
5.5.5	Frontend synthesis observation	26
6	Conclusion	27
6.1	Achievements	27
6.2	Difficulties encountered	27
6.3	Future development	28
6.4	Work distribution	28

Chapter 1

Introduction

Project GitHub Repository

<https://github.com/novar2k4/Vi-mach-Lab.git>

1.1 Project overview

RISC (Reduced Instruction Set Computer) is a processor design philosophy built on the idea of simplifying the instruction set: each instruction performs a single elementary operation, has a fixed length, and a predictable execution time. This approach enables simple pipelining, high clock frequencies, and efficient silicon implementation — and is the foundation of nearly all modern architectures such as ARM, RISC-V, MIPS and PowerPC.

In this project, our group designs a minimal RISC processor with the following specifications:

- 8-bit instruction word, composed of a 3-bit opcode and a 5-bit operand field.
- An instruction set of 8 basic operations: `HLT`, `SKZ`, `ADD`, `AND`, `XOR`, `LDA`, `STO`, `JMP`.
- A 32-byte memory address space (5-bit address) holding both program and data.
- Synchronous operation driven by a `clk` signal, with an active-high `rst`.
- Execution stops on a `HLT` instruction.

1.2 Tools and equipment

1.2.1 Software

- **Cadence Xcelium** / **NCSim** — functional simulation of each Verilog module and the integrated CPU.
- **Cadence Genus** — RTL-to-gate synthesis with the 45 nm GPDK standard-cell library; used for resource and timing evaluation.
- **Cadence SimVision** / **GTKWave** — waveform inspection.

- **Icarus Verilog** (`iverilog`) — a free open-source simulator used for quick functional checks during development.
- **Visual Studio Code** with the Verilog-HDL/SystemVerilog extension for editing.
- **Git** / **GitHub** for source-code version control.

1.2.2 Hardware

- Personal computers of group members (Linux/Windows).
- Departmental SSH server hosting the Cadence toolchain.

1.3 Product features

The CPU implemented in this project is capable of:

1. Fetching instructions from memory at the address held by the Program Counter.
2. Decoding instructions through their 3-bit opcode.
3. Fetching operands from memory when needed (5-bit operand field).
4. Executing the eight ISA operations: halt (**HLT**), conditional skip (**SKZ**), addition (**ADD**), bitwise AND (**AND**), bitwise XOR (**XOR**), accumulator load (**LDA**), accumulator store (**STO**), and unconditional jump (**JMP**).
5. Writing results back to either the Accumulator or to memory.
6. Repeating the fetch–decode–execute cycle until a **HLT** instruction is reached.

Chapter 2

Background

2.1 The RISC philosophy

Unlike CISC (Complex Instruction Set Computer) designs that aim to provide rich, multi-step instructions, RISC architectures follow a few guiding principles:

- A small, uniform instruction set: every instruction has the same width, making decoding straightforward.
- Load–store architecture: only **LDA** / **STO** touch memory; arithmetic and logic operations are register-only.
- Many general-purpose registers: prefer register access over memory access whenever possible.
- Pipelined execution: dividing each instruction into regular stages (fetch, decode, execute, ...) enables instruction-level parallelism.

Because of the strict bit budget (3-bit opcode, 5-bit operand) we adopt an accumulator-based architecture — a single accumulator register is the implicit source/destination for almost every operation. This keeps the core minimal while still illustrating all the major RISC concepts.

2.2 Instruction execution model

A complete instruction cycle in our CPU consists of the following phases:

1. **Instruction Fetch**: the byte at the address pointed to by PC is read from memory and latched into the Instruction Register (IR).
2. **Instruction Decode**: opcode and operand fields are extracted from the IR.
3. **Operand Fetch** (when needed): for **ADD**, **AND**, **XOR** and **LDA**, the operand field is used as an address and the data byte is read from memory.
4. **Execute**: the ALU performs the operation.
5. **Write Back**: the result is written to the Accumulator (**ADD**, **AND**, **XOR**, **LDA**) or to memory (**STO**).

6. **Update PC:** PC is incremented, or loaded with a new value (JMP, or SKZ when the condition holds).

2.3 Real-world relevance

Although this CPU is intentionally minimal, the underlying principles appear in widely deployed processor families:

- **Embedded microcontrollers:** AVR (Microchip), MSP430 (Texas Instruments) and the entire ARM Cortex-M0/M0+ line all adopt a fixed-length, load-store RISC ISA.
- **Low-power IoT devices:** the ARM Thumb subset uses a compact instruction encoding that mirrors the design approach taken here.
- **Open-source RISC-V:** the most rapidly growing instruction set today, increasingly seen in edge-AI SoCs, wearable medical devices and satellite-grade processors.
- **Education:** processors of this size (often based on Mano's Basic Computer) are a standard teaching tool for computer-architecture courses worldwide.

Chapter 3

System Design

3.1 Top-level block diagram

The CPU is composed of seven functional blocks, connected through an 8-bit data bus and a 5-bit address bus. The Controller is a finite state machine (FSM) that distributes control signals to all other blocks to orchestrate each machine cycle.

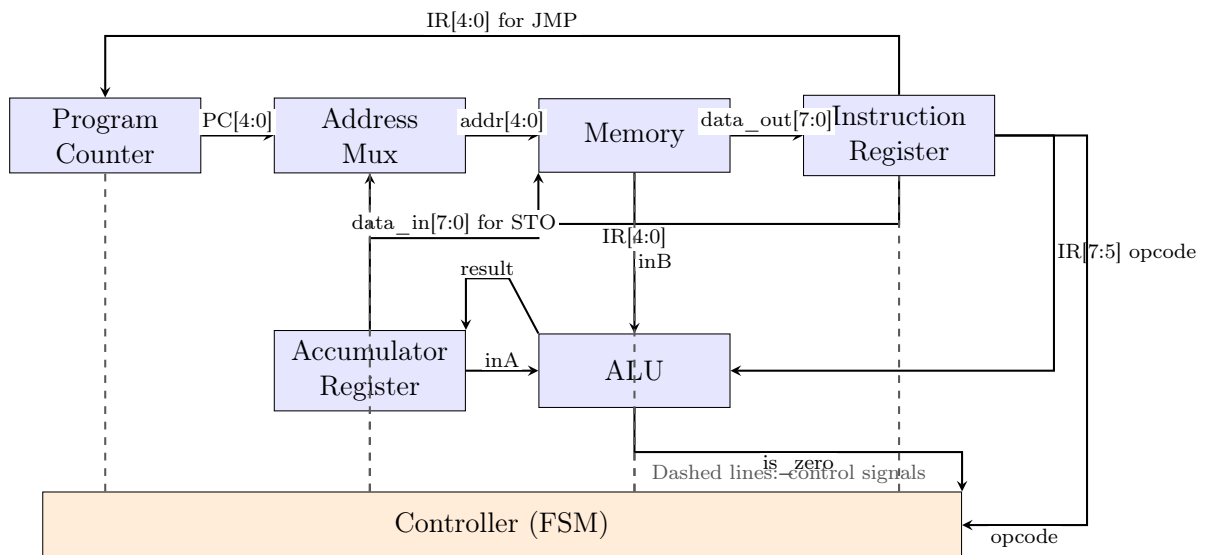


Figure 3.1: Top-level block diagram of the implemented simple RISC CPU

3.2 Functional blocks

3.2.1 Program Counter (PC)

Function: stores the address of the next instruction to be fetched.

- Width: 5 bits (sufficient to address 32 memory locations).
- Operates on the rising edge of `clk`.
- An active-high `rst` forces $PC \leftarrow 0$.
- Asserting `inc_pc` performs $PC \leftarrow PC + 1$.

- Asserting `ld_pc` loads a new value from `IR[4:0]` into the PC, used by `JMP`.
- For a taken `SKZ`, the controller asserts `inc_pc` in `IDLE` and uses `skip_reg` to suppress the next normal increment.

3.2.2 Address Mux

Function: selects which address is presented to memory:

- During instruction fetch: select PC.
- During operand fetch / store: select the operand field of the IR.
- Selector signal: `sel` (1: PC, 0: IR operand).

3.2.3 Memory

Function: stores both program and data within a single address space (Von Neumann organisation).

- 32 entries $\times$ 8 bits.
- Synchronous read and write on the rising edge of `clk`.
- Read enable `rd`, write enable `wr`; the controller guarantees that both are never asserted in the same cycle. The memory module itself does not arbitrate the two signals — it relies on the controller to keep them mutually exclusive.

Although the original specification describes a single bidirectional data port, this implementation models memory reads and writes using separate `data_out` and `data_in` signals for RTL clarity and easier debugging. Functionally, the controller still enforces single-port behavior by never asserting `rd` and `wr` simultaneously.

3.2.4 Instruction Register (IR)

Function: latches the 8-bit instruction word read from memory when `ld_ir` is asserted. The high three bits `IR[7:5]` form the opcode delivered to the Controller and the ALU; the low five bits `IR[4:0]` form the operand field delivered to the Address Mux and to PC (for `JMP`).

3.2.5 Accumulator Register (ACC)

Function: an 8-bit register that captures the ALU output when `ld_ac` is asserted. `ACC` is the implicit source for the `inA` input of the ALU and is also the data source for `ST0`. `ACC` honours the synchronous reset and is cleared to `8'h00` when `rst` is asserted.

3.2.6 ALU

Function: performs one of the eight operations selected by the 3-bit opcode field. Inputs: `inA` (8 bits, from `ACC`) and `inB` (8 bits, from memory). Outputs: `result` (8 bits) and `is_zero` (1 bit, asynchronous flag indicating that the combinational result is zero, used by `SKZ`).

Table 3.1: Instruction set and ALU behaviour

Opcode	Binary	Operation	Result
HLT	000	Stop the program	inA
SKZ	001	If inA = 0, skip the next instruction	inA
ADD	010	ACC + Mem[operand] $\rightarrow$ ACC	inA + inB
AND	011	ACC AND Mem[operand] $\rightarrow$ ACC	inA AND inB
XOR	100	ACC XOR Mem[operand] $\rightarrow$ ACC	inA XOR inB
LDA	101	Mem[operand] $\rightarrow$ ACC	inB
STO	110	ACC $\rightarrow$ Mem[operand]	inA
JMP	111	PC $\leftarrow$ operand	inA

3.2.7 Controller

Function: an FSM that generates the control signals based on the current state and the opcode.

- **Eight states:** INST_ADDR, INST_FETCH, INST_LOAD, IDLE, OP_ADDR, OP_FETCH, ALU_OP, STORE.
- **Reset state:** INST_ADDR.
- **One internal flag:** skip_reg, a single-bit register that records whether the previous SKZ took its skip path. When set, it suppresses the inc_pc pulse normally generated by INST_LOAD; the flag is then cleared on the next INST_ADDR. This is how the SKZ instruction skips the following instruction without disturbing PC bookkeeping (see Section 6.2).
- **Generates nine control outputs:** sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e. The data_e signal is asserted in STORE alongside wr; in the present revision it is exposed at the controller interface but is not consumed downstream (the memory uses wr only). It is retained so that a future revision could gate a tristate data bus or a write-enable to an external port.
- **Conditional state transitions** short-circuit the pipeline when an instruction does not need every phase: for instance HLT stays in IDLE, JMP and SKZ return directly from IDLE to INST_ADDR, and only STO reaches the STORE phase.

Table 3.2: Control-signal table by state and opcode

Output	INST_ADDR	INST_FETCH	INST_LOAD	IDLE	OP_ADDR	OP_FETCH	ALU_OP	STORE
sel	1	1	1	0	0	0	0	0
rd	1	1	0	0	1	1	0	0
ld_ir	0	0	1	0	0	0	0	0
halt	0	0	0	HLT	0	0	0	0
inc_pc	0	0	$\sim$ skip_reg	SKZ&zero	0	0	0	0
ld_ac	0	0	0	0	0	0	1	0
ld_pc	0	0	0	JMP	0	0	0	0
wr	0	0	0	0	0	0	0	1
data_e	0	0	0	0	0	0	0	1

The conditional entries follow the natural rule of the FSM: halt pulses only when IDLE sees HLT; ld_pc pulses only when IDLE sees JMP; inc_pc fires in INST_LOAD for every

instruction *unless* the previous instruction was a taken SKZ (in which case `skip_reg` is high and the increment is suppressed for one cycle), and additionally fires once in IDLE when the active opcode is SKZ and the zero flag is asserted (this is the increment that effectively *skips* over the following instruction's fetch address).

Chapter 4

Implementation

4.1 Project layout

The source code is organised as a flat hierarchy with three Verilog files plus a Python helper:

```
risc-cpu/  
  src.v          # Top module + all 7 functional modules  
  tb.v           # Top-level testbench (loads init.mem)  
  tb_modules.v   # Per-module unit tests  
  convert.py     # Assembly-listing -> $readmemb format  
  prog1.txt      # Test 1: JMP / LDA / SKZ / ST0 / XOR  
  prog2.txt      # Test 2: AND / ADD  
  prog3.txt      # Test 3: Fibonacci sequence up to 144
```

The Python helper reads the project's listing format (which carries @addr markers for non-contiguous regions and underscore-separated binary literals like 111_11110) and emits a flat 32-line file suitable for \$readmemb, with all unspecified locations filled with zeros.

4.1.1 Source code repository

The complete project source code, including RTL modules, testbenches, simulation programs, synthesis scripts and report sources, is maintained in a GitHub repository for version control and collaborative development.

GitHub Repository:

<https://github.com/novar2k4/Vi-mach-Lab.git>

The repository was used throughout the project for source tracking, team collaboration, debugging and experiment management.

4.2 Verilog modules

4.2.1 Top-level wrapper

The top-level `risc_cpu` module instantiates the seven functional blocks and wires them together. A small set of debug outputs (PC, ACC, IR, FSM state, `halt`) is exported to make the testbench simpler. Note that `debug_state` is declared as a 3-bit port and is

driven by zero-extending the controller's internal state register; the high-order zeros are dropped by the simulator port mapping.

```

1 module risc_cpu (
2     input  wire clk, rst,
3     output wire [7:0] debug_pc, debug_acc, debug_ir,
4     output wire [2:0] debug_state,
5     output wire debug_halt
6 );
7     wire [4:0] pc_out, addr_mux_out;
8     wire [7:0] mem_data_out, ir_out, acc_out, alu_out;
9     wire alu_zero, mem_rd, mem_wr;
10    wire sel, ld_ir, halt, inc_pc, ld_ac, ld_pc, data_e;
11
12    program_counter    pc      (.clk(clk), .rst(rst), .inc(inc_pc),
13                                .load(ld_pc), .data_in(ir_out[4:0]),
14                                .count(pc_out));
15    address_mux        addr_mux (.sel(sel), .pc_addr(pc_out),
16                                .ir_addr(ir_out[4:0]),
17                                .addr_out(addr_mux_out));
18    memory             mem      (.clk(clk), .addr(addr_mux_out),
19                                .data_in(acc_out), .data_out(
20                                    mem_data_out),
21                                .rd(mem_rd), .wr(mem_wr));
22    instruction_register ir      (.clk(clk), .data_in(mem_data_out),
23                                .load(ld_ir), .data_out(ir_out));
24    accumulator        acc      (.clk(clk), .rst(rst), .data_in(
25                                    alu_out),
26                                    .load(ld_ac), .data_out(acc_out));
27    alu                alu_unit (.inA(acc_out), .inB(mem_data_out),
28                                .opcode(ir_out[7:5]),
29                                .result(alu_out), .is_zero(alu_zero))
30                                ;
31    controller         ctrl     (.clk(clk), .rst(rst),
32                                .opcode(ir_out[7:5]), .zero(alu_zero)
33                                ,
34                                .sel(sel), .rd(mem_rd), .ld_ir(ld_ir)
35                                ,
36                                .halt(halt), .inc_pc(inc_pc),
37                                .ld_ac(ld_ac), .ld_pc(ld_pc),
38                                .wr(mem_wr), .data_e(data_e));
39
40    assign debug_pc     = {3'b000, pc_out};
41    assign debug_acc    = acc_out;
42    assign debug_ir     = ir_out;
43    assign debug_state  = {2'b00, ctrl.state};
44    assign debug_halt   = halt;
45 endmodule

```

Listing 4.1: Top-level instantiation (excerpt from src.v)

4.2.2 Program Counter

```

1 module program_counter (
2     input  wire clk, rst,
3     input  wire inc, load,
4     input  wire [4:0] data_in,

```

```

5     output reg  [4:0] count
6 );
7     always @(posedge clk) begin
8         if (rst)      count <= 5'b00000;
9         else if (load) count <= data_in;
10        else if (inc)  count <= count + 1'b1;
11    end
12 endmodule

```

Listing 4.2: Program Counter

4.2.3 Address Mux

```

1 module address_mux (
2     input  wire sel,
3     input  wire [4:0] pc_addr, ir_addr,
4     output wire [4:0] addr_out
5 );
6     assign addr_out = sel ? pc_addr : ir_addr;
7 endmodule

```

Listing 4.3: Address Mux (2-input)

4.2.4 Memory

```

1 module memory (
2     input  wire clk,
3     input  wire [4:0] addr,
4     input  wire [7:0] data_in,
5     output reg [7:0] data_out,
6     input  wire rd, wr
7 );
8     reg [7:0] mem [0:31];
9
10    always @(posedge clk) begin
11        if (rd) data_out <= mem[addr];
12        if (wr) mem[addr] <= data_in;
13    end
14 endmodule

```

Listing 4.4: Memory — 32×8 -bit

4.2.5 Instruction Register and Accumulator

The IR is a simple gated 8-bit register that latches `data_in` on the next rising edge whenever `load` is asserted; it has no reset. The Accumulator is the same shape but additionally honours a synchronous reset that clears the register to zero.

```

1 module instruction_register (
2     input  wire clk,
3     input  wire [7:0] data_in,
4     input  wire load,
5     output reg [7:0] data_out
6 );

```

```

7     always @(posedge clk) begin
8         if (load) data_out <= data_in;
9     end
10 endmodule

```

Listing 4.5: Instruction Register

```

1 module accumulator (
2     input  wire clk, rst,
3     input  wire [7:0] data_in,
4     input  wire load,
5     output reg  [7:0] data_out
6 );
7     always @(posedge clk) begin
8         if (rst) data_out <= 8'h00;
9         else if (load) data_out <= data_in;
10    end
11 endmodule

```

Listing 4.6: Accumulator

4.2.6 ALU

```

1 module alu (
2     input  wire [7:0] inA, inB,
3     input  wire [2:0] opcode,
4     output reg  [7:0] result,
5     output wire is_zero
6 );
7     assign is_zero = (result == 8'h00) ? 1'b1 : 1'b0;
8
9     always @(*) begin
10        case (opcode)
11            3'b000: result = inA;           // HLT
12            3'b001: result = inA;           // SKZ
13            3'b010: result = inA + inB;     // ADD
14            3'b011: result = inA & inB;     // AND
15            3'b100: result = inA ^ inB;     // XOR
16            3'b101: result = inB;           // LDA
17            3'b110: result = inA;           // STO
18            3'b111: result = inA;           // JMP
19            default: result = 8'h00;
20        endcase
21    end
22 endmodule

```

Listing 4.7: ALU

4.2.7 Controller

The Controller is the most intricate module. It comprises three `always` blocks: a sequential state register (which also maintains the `skip_reg` flag), a combinational next-state function, and a combinational output decoder.

```

1 module controller (
2     input  wire clk, rst,

```

```

3  input  wire [2:0] opcode,
4  input  wire zero,
5  output reg  sel, rd, ld_ir,
6  output reg  halt, inc_pc, ld_ac,
7  output reg  ld_pc, wr, data_e
8  );
9
10 parameter [2:0] INST_ADDR=0, INST_FETCH=1, INST_LOAD=2, IDLE=3,
11                OP_ADDR=4, OP_FETCH=5, ALU_OP=6, STORE=7;
12
13 reg [2:0] state, next;
14 reg      skip_reg;      // 1 cycle after a taken SKZ
15
16 // 1) State register + skip flag
17 always @(posedge clk) begin
18     if (rst) begin
19         state    <= INST_ADDR;
20         skip_reg <= 1'b0;
21     end else begin
22         state <= next;
23         if (state == IDLE && opcode == 3'b001 && zero)
24             skip_reg <= 1'b1;      // SKZ taken: arm suppression
25         else if (state == INST_ADDR && skip_reg)
26             skip_reg <= 1'b0;      // consumed at next INST_ADDR
27     end
28 end
29
30 // 2) Next-state logic (conditional skips of unused phases)
31 always @(*) begin
32     case (state)
33         INST_ADDR : next = INST_FETCH;
34         INST_FETCH: next = INST_LOAD;
35         INST_LOAD  : next = IDLE;
36         IDLE       : next = (opcode==3'b000) ? IDLE :
37                             (opcode==3'b111 || opcode==3'b001)
38                             ? INST_ADDR : OP_ADDR;
39         OP_ADDR    : next = OP_FETCH;
40         OP_FETCH   : next = ALU_OP;
41         ALU_OP     : next = (opcode==3'b110) ? STORE : INST_ADDR;
42         STORE      : next = INST_ADDR;
43         default    : next = INST_ADDR;
44     endcase
45 end
46
47 // 3) Output decoder (Moore-style)
48 always @(*) begin
49     sel=1; rd=0; ld_ir=0; halt=0;
50     inc_pc=0; ld_ac=0; ld_pc=0; wr=0; data_e=0;
51     case (state)
52         INST_ADDR : begin sel=1; rd=1; end
53         INST_FETCH: begin sel=1; rd=1; end
54         INST_LOAD  : begin sel=1; ld_ir=1; inc_pc = ~skip_reg; end
55         IDLE       : begin
56             sel=0;
57             if (opcode==3'b000) halt=1;
58             else if (opcode==3'b001 && zero) inc_pc
59                 =1;
59             else if (opcode==3'b111) ld_pc=1;
60         end

```



```

60         OP_ADDR    : begin sel=0; rd=1;                                end
61         OP_FETCH   : begin sel=0; rd=1;                                end
62         ALU_OP      : begin sel=0; ld_ac=1;                             end
63         STORE       : begin sel=0; wr=1; data_e=1;                     end
64     endcase
65 end
66 endmodule

```

Listing 4.8: Controller (FSM)

4.3 State-transition diagram

Figure 4.1 shows the optimized FSM used in our implementation.

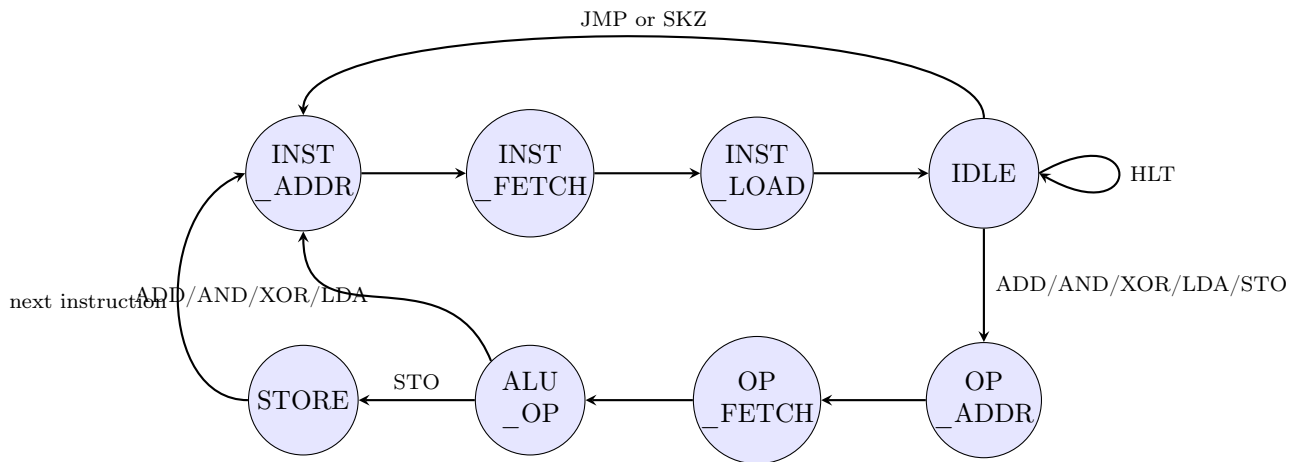


Figure 4.1: Optimized controller state-transition diagram used in this implementation

4.3.1 Instruction timing characteristics

Because the controller is implemented as a multi-cycle finite-state machine (FSM), different instructions require different numbers of clock cycles depending on their execution path through the controller states.

The latency is counted from the first instruction-address state (INST_ADDR) until the state where the instruction completes its main architectural effect.

Table 4.1 summarizes the execution latency of each instruction class.

Table 4.1: Instruction execution latency

Instruction	FSM path	Cycles
HLT	INST_ADDR → INST_FETCH → INST_LOAD → IDLE	4
SKZ	INST_ADDR → INST_FETCH → INST_LOAD → IDLE	4
JMP	INST_ADDR → INST_FETCH → INST_LOAD → IDLE	4
ADD / AND / XOR / LDA	INST_ADDR → INST_FETCH → INST_LOAD → IDLE → OP_ADDR → OP_FETCH → ALU_OP	7
STO	INST_ADDR → INST_FETCH → INST_LOAD → IDLE → OP_ADDR → OP_FETCH → ALU_OP → STORE	8

The control-flow instructions (HLT, SKZ and JMP) complete immediately after the instruction decode stage and therefore require only four cycles. Arithmetic, logical and load operations additionally require operand-fetch and ALU-execution phases, increasing the latency to seven cycles. The STO instruction requires an extra dedicated memory-write state and therefore has the largest latency of eight cycles.

4.3.2 Cycles per instruction (CPI)

Because the processor uses a multi-cycle FSM controller, the number of clock cycles per instruction depends on the instruction type and the required execution stages.

Control-flow instructions such as HLT, SKZ and JMP complete after instruction fetch and decode, resulting in a CPI of 4. Arithmetic and logical instructions require operand fetch and ALU execution, increasing the CPI to 7. The STO instruction additionally performs a memory-write stage and therefore requires 8 cycles.

Assuming a balanced instruction distribution, the average CPI of the processor is approximately between 5 and 7 cycles per instruction. Although this is slower than modern pipelined RISC processors, the multi-cycle approach greatly simplifies the controller and datapath implementation, making it suitable for an educational frontend RTL design project.

4.4 Per-instruction execution flow

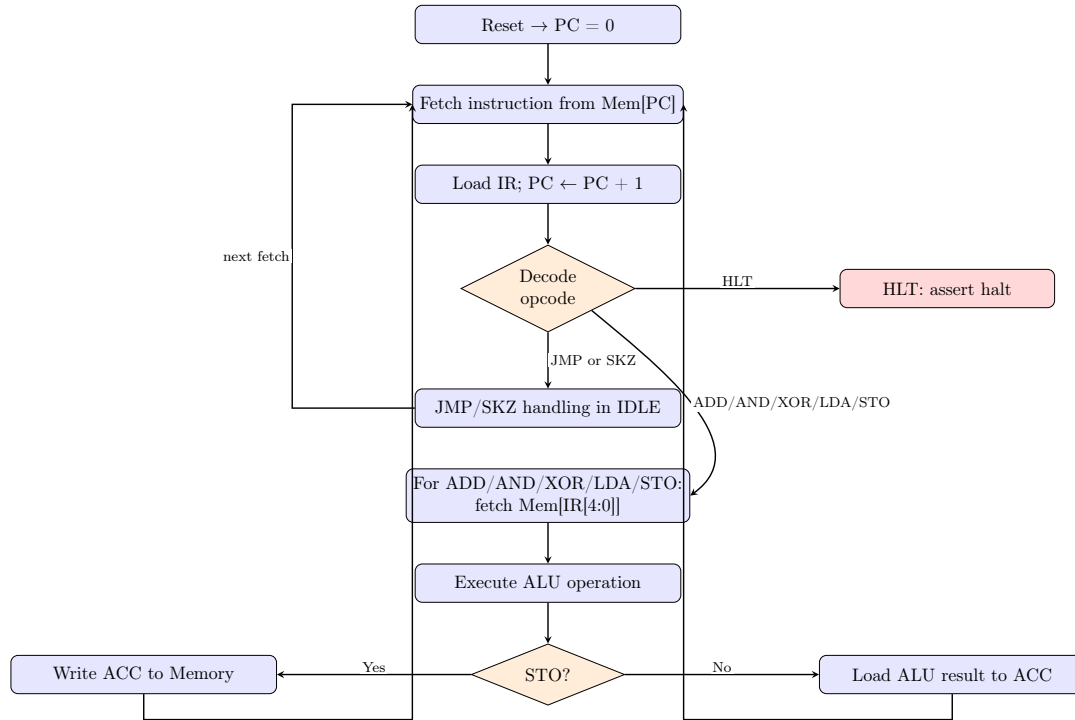


Figure 4.2: Per-instruction execution flow matching the implemented controller

4.5 Assembler helper

Test programs are written in a lightweight listing format that mirrors the assignment specification (binary literals with underscore separators, an `@addr` directive for non-contiguous regions). The Python helper `convert.py` consumes this format and emits a 32-line memory-image file suitable for `$readmemb`, with all unspecified addresses zero-filled:

```

1 import sys
2
3 def main():
4     with open(sys.argv[1], "r", encoding="utf-8") as f:
5         lines = f.readlines()
6         result = [None] * 32
7         current_addr = 0
8         for line in lines:
9             line = line.strip()
10            if not line or line.startswith("//"):
11                continue
12            if line.startswith("@"):
13                parts = line.split()
14                current_addr = int(parts[0][1:], 16)
15                if len(parts) > 1:
16                    code = parts[1].replace("_", "")
17                    if len(code) == 8 and all(c in "01" for c in code):
18                        result[current_addr] = code
19                        current_addr += 1

```

```
20         continue
21     code = line.split()[0].replace("_", "")
22     if len(code) == 8 and all(c in "01" for c in code):
23         result[current_addr] = code
24         current_addr += 1
25     for b in result:
26         print(b if b is not None else "00000000")
27
28 if __name__ == "__main__":
29     main()
```

Listing 4.9: convert.py

Chapter 5

Verification and Evaluation

5.1 Verification methodology

Verification is performed at two levels:

- **Module-level (unit) tests** in `tb_modules.v`: each block is exercised standalone with directed stimulus and the result is compared against the expected value via `$display`. The unit suite covers PC counting and loading, mux selection, memory write/read, IR latching, ACC loading, three ALU operations (ADD, XOR, zero flag) and a Controller halt sequence.
- **System-level tests** in `tb.v`: the integrated `risc_cpu` runs three programs loaded via `$readmemb` into the memory's internal array. The testbench prints PC, IR, ACC, FSM state, the zero flag, the ALU's `inA/inB/result`, and the memory `data_out` on every clock edge, and exits when `halt` is asserted, reporting the final PC value.

5.2 Test program 1 — JMP / LDA / SKZ / STO / XOR

Program 1 walks through the control-flow and data-movement instructions. Each step is gated by a SKZ or JMP that re-routes execution to a halting error message at the first failure; reaching the HLT at 0x17 indicates that every checked instruction works correctly.

```
000 111_11110    // 00  BEGIN:    JMP TST_JMP
    000_00000    // 01          HLT          // JMP did not work at all
    000_00000    // 02          HLT          // JMP skipped a cycle
    101_11010    // 03  JMP_OK:  LDA DATA_1
    001_00000    // 04          SKZ
    000_00000    // 05          HLT          // SKZ or LDA broken
    101_11011    // 06          LDA DATA_2
    001_00000    // 07          SKZ
    111_01010    // 08          JMP SKZ_OK
    000_00000    // 09          HLT
    110_11100    // 0A  SKZ_OK:  STO TEMP    // store non-zero value
    101_11010    // 0B          LDA DATA_1
    110_11100    // 0C          STO TEMP    // overwrite with zero
    101_11100    // 0D          LDA TEMP
    001_00000    // 0E          SKZ
    000_00000    // 0F          HLT          // STO did not work
    100_11011    // 10          XOR DATA_2
    001_00000    // 11          SKZ
    111_10100    // 12          JMP XOR_OK
    000_00000    // 13          HLT
```

```

100_11011    // 14  XOR_OK:  XOR DATA_2
001_00000    // 15          SKZ
000_00000    // 16          HLT
000_00000    // 17  END:    HLT          // PASSED
111_00000    // 18          JMP BEGIN
@1A 00000000  // 1A  DATA_1 = 0x00
11111111    // 1B  DATA_2 = 0xFF
10101010    // 1C  TEMP  = 0xAA
@1E 111_00011  // 1E  TST_JMP: JMP JMP_OK
000_00000    // 1F          HLT

```

Listing 5.1: prog1.txt — ISA functional check

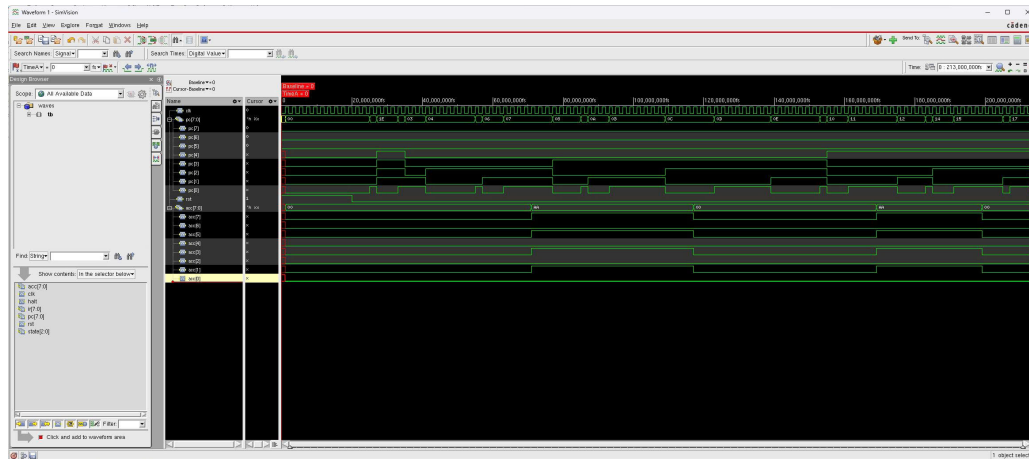
Expected outcome: the simulator reports `halt` when the program reaches `HLT` at `PC = 0x17`.

Observed outcome: the simulator halts with `PC = 0x18` (post-fetch increment), confirming the executed instruction was the `HLT` at `0x17`. The end-of-run banner is reproduced below; the full instruction-by-instruction trace is omitted for brevity.

```

=====
END PROGRAM
PC=18  Acc=00(0)  Zero=1
=====

```



5.3 Test program 2 — AND / ADD

Program 2 verifies the bitwise-AND and integer-addition data-paths. The chain of computations is constructed so that every individual result must equal zero (and therefore force `SKZ` to skip the guard `HLT`); reaching `HLT` at `0x10` signals success.

```

@00 101_11011    // 00  BEGIN:  LDA DATA_2 ; acc = 0xAA
011_11100    // 01          AND DATA_3 ; 0xAA & 0xFF = 0xAA
100_11011    // 02          XOR DATA_2 ; ~ 0xAA = 0x00
001_00000    // 03          SKZ
000_00000    // 04          HLT          ; AND broken
010_11010    // 05          ADD DATA_1 ; acc += 1
001_00000    // 06          SKZ
111_01001    // 07          JMP ADD_OK
000_00000    // 08          HLT          ; ADD broken
100_11100    // 09          XOR DATA_3 ; sign-flip via XOR FF
010_11010    // 0A          ADD DATA_1 ; result = 0xFF (-1)
110_11101    // 0B          STO TEMP
101_11010    // 0C          LDA DATA_1
010_11101    // 0D          ADD TEMP    ; 1 + (-1) = 0
001_00000    // 0E          SKZ
000_00000    // 0F          HLT
000_00000    // 10  END:    HLT          ; PASSED
111_00000    // 11          JMP BEGIN

```

```

@1A 00000001 // 1A DATA_1 = 0x01
      10101010 // 1B DATA_2 = 0xAA
      11111111 // 1C DATA_3 = 0xFF
      00000000 // 1D TEMP

```

Listing 5.2: prog2.txt — AND/ADD check

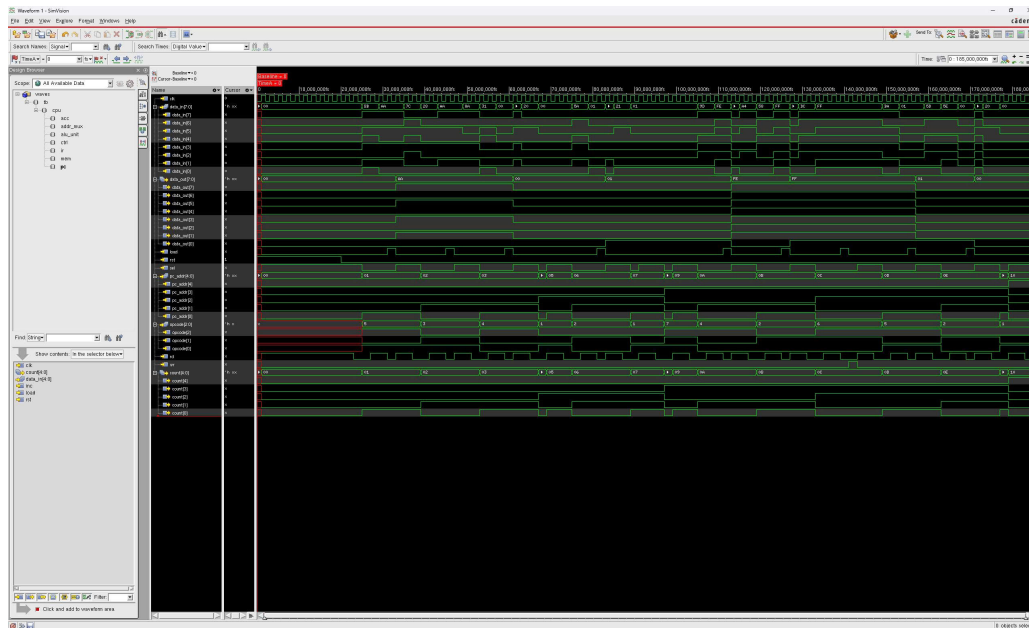
Expected outcome: HLT at PC = 0x10.

Observed outcome: the simulator halts at PC = 0x11 (post-fetch increment), confirming the executed instruction was the PASSED HLT at 0x10.

```

=====
END PROGRAM
PC=11 Acc=00(0) Zero=1
=====

```



5.4 Test program 3 — Fibonacci

Program 3 computes the Fibonacci sequence and stops when the running value reaches the limit 0x90 (= 144 decimal). It exercises LDA, STO, ADD, XOR, SKZ and JMP simultaneously and so functions as an end-to-end soak test.

```

      111_00011 // 00          JMP LOOP    ; bypass AGAIN block
@03  101_11011 // 03  LOOP:    LDA FN2
      110_11100 // 04          STO TEMP
      010_11010 // 05          ADD FN1      ; FN2 + FN1
      110_11011 // 06          STO FN2      ; new FN2
      101_11100 // 07          LDA TEMP
      110_11010 // 08          STO FN1      ; new FN1 = old FN2
      100_11101 // 09          XOR LIMIT
      001_00000 // 0A          SKZ          ; if FN1 == 144, exit
      111_00011 // 0B          JMP LOOP
      000_00000 // 0C  DONE:    HLT
      101_11111 // 0D  AGAIN:   LDA ONE      ; (re-init block, unused
      110_11010 // 0E          STO FN1      ; in this run)
      101_11110 // 0F          LDA ZERO
      110_11011 // 10          STO FN2
      111_00011 // 11          JMP LOOP
@1A  00000001 // 1A  FN1 = 1
      00000000 // 1B  FN2 = 0
      00000000 // 1C  TEMP
      10010000 // 1D  LIMIT = 144

```

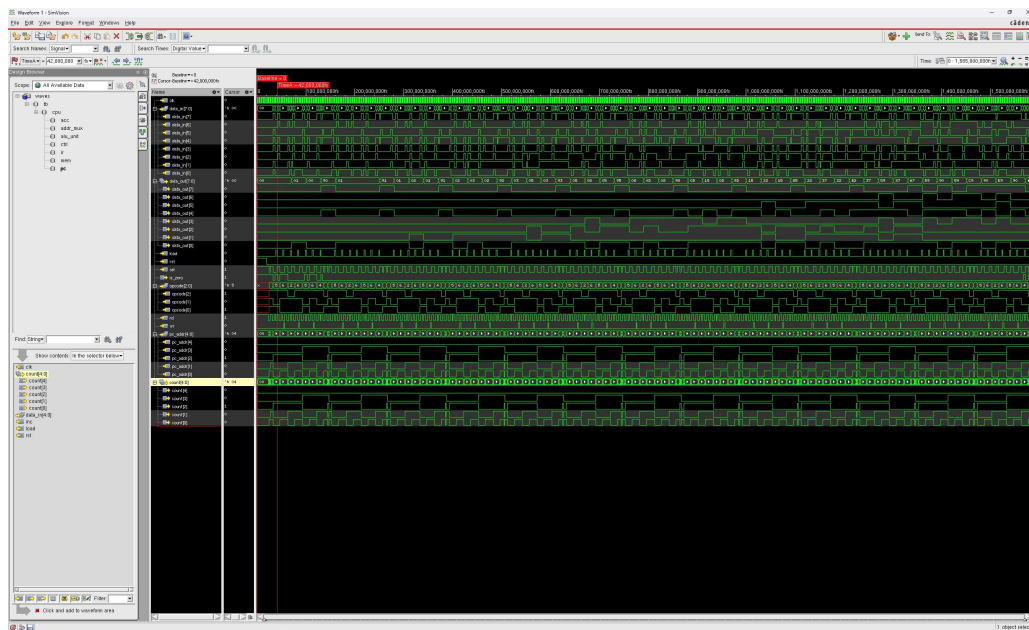
```
00000000    // 1E  ZERO  = 0
00000001    // 1F  ONE   = 1
```

Listing 5.3: prog3.txt — Fibonacci

Expected outcome: HLT at PC = 0x0C once the running Fibonacci value reaches 0x90.

Observed outcome: the simulator halts at PC = 0x0D (post-fetch increment), confirming the executed instruction was the HLT at 0x0C. Inspection of the Accumulator trajectory shows the expected sequence 0x01, 0x01, 0x02, 0x03, 0x05, 0x08, 0x0D, 0x15, 0x22, 0x37, 0x59, 0x90 (1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144).

```
=====
END PROGRAM
PC=0d  Acc=90(144)  Zero=0
=====
```



5.5 Performance evaluation

This project focuses on the frontend digital-design flow, including RTL implementation, functional simulation, logic synthesis, and timing analysis. The backend physical-design stages such as floorplanning, placement, routing, clock-tree synthesis and signoff are outside the scope of this work.

5.5.1 Synthesis setup

The RTL design was synthesized using Cadence Genus. The synthesis script loads the Verilog source file, applies the timing constraints, runs generic synthesis, technology mapping and optimization, and finally generates the QoR, timing, area, power and gate-level netlist reports.

```
read_hdl "src.v"
elaborate $DESIGN
read_sdc risc_cpu.sdc

syn_gen
```



```

syn_map
syn_opt

report_qor      > ${_OUTPUTS_PATH}/qor.rpt
report_timing  > ${_OUTPUTS_PATH}/timing.rpt
report_area     > ${_OUTPUTS_PATH}/area.rpt
report_power    > ${_OUTPUTS_PATH}/power.rpt

write_hdl > ${_OUTPUTS_PATH}/${DESIGN}_m.v

```

Listing 5.4: Main synthesis flow in `run.tcl`

The target clock period is set to 1 ns in the SDC file, corresponding to a target frontend frequency of 1 GHz.

```

create_clock -name "clk" -add -period 1 -waveform {0.0 0.5} [get_ports clk]

set_input_delay  -clock [get_clocks clk] -add_delay 0.5 [get_ports rst]
set_output_delay -clock [get_clocks clk] -add_delay 0.5 [get_ports debug_pc]
set_output_delay -clock [get_clocks clk] -add_delay 0.5 [get_ports debug_acc]
set_output_delay -clock [get_clocks clk] -add_delay 0.5 [get_ports debug_ir]
set_output_delay -clock [get_clocks clk] -add_delay 0.5 [get_ports debug_state]
set_output_delay -clock [get_clocks clk] -add_delay 0.5 [get_ports debug_halt]

set_max_fanout 15.000 [current_design]
set_max_transition 1.2 [current_design]

```

Listing 5.5: Clock constraint in `risc_cpu.sdc`

5.5.2 Area result

Table 5.1: Area report summary

Instance	Cell Count	Cell Area	Net Area	Total Area
risc_cpu	27	78.660	35.237	113.897
ctrl	9	18.810	4.428	23.238

The synthesized design contains 27 standard-cell instances. The total reported area is 113.897, including 78.660 cell area and 35.237 net area. The controller accounts for 9 cells and 23.238 total area, showing that the control FSM is a small but important part of the design.

5.5.3 Timing result

Table 5.2: Timing and QoR summary

Metric	Value
Clock period constraint	1000 ps / 1 ns
Target frequency	1 GHz
Critical path slack	78 ps
Total negative slack (TNS)	0.0 ps
Violating paths	0
Leaf instance count	27
Sequential instance count	7
Combinational instance count	20
Hierarchical instance count	1
Maximum fanout	19
Average fanout	2.4

The timing report shows that the design meets the 1 ns clock constraint. The critical path has a positive slack of 78 ps, with zero violating paths and zero total negative slack. Therefore, the frontend synthesized design satisfies the target 1 GHz timing requirement.

The critical setup path reported by Genus starts from the reset input and ends at the D input of `pc_count_reg[4]`. The required time is 965 ps, while the data arrival time is 887 ps, resulting in 78 ps of positive slack.

```
Path 1: MET (78 ps) Setup Check with Pin pc_count_reg[4]/CK->D

Clock Edge      : 1000 ps
Setup Time      : 35 ps
Required Time    : 965 ps
Input Delay      : 500 ps
Data Path Delay  : 387 ps
Arrival Time     : 887 ps
Slack           : 78 ps
```

Listing 5.6: Critical timing path summary

5.5.4 Power result

Table 5.3: Power report summary

Category	Leakage (W)	Internal (W)	Switching (W)	Total (W)
Register	5.83725×10^{-9}	9.43032×10^{-5}	1.55789×10^{-5}	1.09888×10^{-4}
Logic	3.58365×10^{-9}	1.04637×10^{-5}	2.52793×10^{-5}	3.57465×10^{-5}
Clock	0	0	8.01504×10^{-6}	8.01504×10^{-6}
Total	9.42090×10^{-9}	1.04767×10^{-4}	4.88733×10^{-5}	1.53650×10^{-4}

The total estimated power is 1.53650×10^{-4} W. Register power is the dominant component, contributing 71.52% of the total power. Logic contributes 23.26%, while clock

switching contributes 5.22%. Leakage power is negligible for this small synthesized design, accounting for only 0.01% of the total.

5.5.5 Frontend synthesis observation

The generated gate-level netlist confirms that Cadence Genus successfully mapped the RTL design into technology-library standard cells such as flip-flops, inverters, NAND/NOR gates and AOI/OAI logic gates.

Because this project focuses on frontend RTL synthesis rather than a full post-layout implementation, the synthesis tool performs aggressive logic optimization on signals that have limited architectural visibility at the top level. As a result, the synthesized design becomes significantly more compact than the behavioral RTL description while still preserving the required functional behavior and timing constraints.

Table 5.2 shows that the synthesized design successfully satisfies the 1 GHz target clock requirement with positive timing slack and zero violating paths.

An excerpt from the synthesized controller netlist is shown below.

```
1 DFFHQX1 \state_reg[1] (.CK(clk), .D(n_10), .Q(state[1]));
2 DFFHQX1 \state_reg[0] (.CK(clk), .D(n_1), .Q(state[0]));
3
4 INVXL g779(.A(n_6), .Y(halt));
5 NOR2X1 g786(.A(state[0]), .B(n_5), .Y(inc_pc));
```

Listing 5.7: Excerpt from synthesized controller netlist

Chapter 6

Conclusion

6.1 Achievements

Over the course of this project the group has:

- Implemented all seven required Verilog modules (PC, Address Mux, Memory, IR, ACC, ALU and Controller) plus a top-level wrapper.
- Verified each module standalone with a directed test bench and the integrated CPU with three end-to-end programs.
- Designed a small assembler helper that converts the human-readable listing format described in the assignment into a memory image file consumable by `$readmemb`.
- Exercised the standard digital-design flow — specification, modular RTL coding, simulation, and synthesis — on a complete (if minimal) processor.

6.2 Difficulties encountered

- **Pipeline timing of SKZ.** The SKZ instruction must skip the next instruction in memory when its operand is zero. The naive idea — pulse `inc_pc` a second time whenever IDLE sees SKZ with the zero flag set — produces only *one* extra increment on top of the INST_LOAD increment that already happened, so the next byte is fetched from PC+2 instead of being skipped. The next INST_LOAD would then add a further +1, leaving the program counter at the address *after* the one that should run. Our solution is the small `skip_reg` flag in the controller: the moment IDLE sees a taken SKZ, `skip_reg` is set; the *next* INST_LOAD therefore drives `inc_pc = ~skip_reg = 0`, i.e. it reads the byte at PC+1 (the instruction we want to skip past, into the IR — but no longer increments PC over it). `skip_reg` is then cleared at the next INST_ADDR so subsequent instructions advance normally. Without this flag, the instruction following the skipped one would either be re-fetched or skipped twice, depending on the chosen polarity.
- **Single-port memory.** Because the assignment forbids simultaneous read and write on the data bus, the controller had to be carefully arranged so that `rd` and `wr` are never asserted in the same cycle. This is made trivial in the implementation by isolating the STORE state, which is the only place where `wr` pulses; `rd` is asserted only in INST_ADDR, INST_FETCH, OP_ADDR and OP_FETCH.

- **Asynchronous is_zero.** Because the zero flag is purely combinational on the ALU result, care was needed to ensure that the **zero** signal seen by the controller in the **IDLE** state corresponds to the correct **ACC** value. Sampling **zero** together with the opcode in **IDLE** (where the ALU's combinational result equals **ACC** for the **SKZ** opcode) avoids any glitch hazard.
- **Unused control signal data_e.** The specification lists **data_e** alongside **wr** for the **STORE** phase. In our implementation the memory module does not consume **data_e**, but we kept the signal driven so that the control interface matches the specification table and so that a future revision (e.g. a tristate data bus) can pick it up without re-touching the FSM.

6.3 Future development

Possible future improvements include:

- Replacing the behavioral memory with a synthesizable SRAM macro or a single bidirectional data bus interface.
- Extending the ALU with additional operations such as shift, multiply, or compare.
- Adding a small register file to move from an accumulator-based CPU toward a more general RISC datapath.
- Improving the controller to support more regular instruction timing and easier pipelining.
- Running post-synthesis and post-layout simulations to validate timing and power after physical implementation.

6.4 Work distribution

Table 6.1: Work distribution among team members

Member	Student ID	Main contribution
Huynh Chanh Minh	2352736	CPU datapath, report writing
Dang Thai Khang	2352464	Controller FSM, SKZ/JMP handling
Tran Nguyen Nam Hai	2252190	Testbench, simulation, waveform checking
Dang Duy Nguyen	2352821	Synthesis flow, QoR/timing/power reports

Bibliography

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*, Morgan Kaufmann, 2017.
- [2] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.
- [3] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.
- [4] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, latest available specification.
- [5] Cadence Design Systems, *Genus Synthesis Solution User Guide*, Cadence Documentation.
- [6] Cadence Design Systems, *Xcelium Simulator User Guide*, Cadence Documentation.
- [7] LSI Logic Design course slides, Department of Computer Engineering, HCMUT — VNU-HCM, Semester 242.